

2.3 Object-Oriented Programming

Structuring code into functions or methods is only the first step on the way to the well-thought-out development of complex, large programs. It's even more difficult to design code in a meaningful object-oriented way. What belongs to a class? Which data is public, and which is private? What details does the constructor have to take care of? At which (data) boundaries are classes differentiated from one another? Where can mechanisms such as inheritance or interfaces be used appropriately?

As in the previous sections, our aim here isn't to teach you how to program. Nor did we consider it appropriate to print a lot of code here. Our primary goal is to use examples to motivate you to let AI bots or wizards provide you with targeted support. Not only does this work with regard to formulating the code, but also—and even more important—in the correct designing of classes.

Why Java?

The following examples use Java as the programming language. Although you can also define classes in Python, its strengths lie elsewhere. Java offers significantly more object-oriented features.

Admittedly, Java is a fairly old language. You can formulate object-oriented code much more elegantly in new programming languages such as Swift or Kotlin. However, these languages are relatively new, and features change or are added. For this reason, and because of the limited distribution (i.e., the limited availability of training material), language models work better for long-established languages than they do for new languages.

Long story short—even if our love for Java is limited, this widely used language seemed to be the best choice for the following examples.

2.3.1 Example: User Account Class

The following prompts are intended to show how AI tools can help you with the object-oriented design of programs. The first prompts concern a program section in which a user account is represented by a Java class. In the prompt, we summarize the objective of the class, list its most important properties, and name the programming language we're using. Only then does the actual question follow: What kind of validation should be performed in the constructor of the class?

Prompt: I need to write a Java class for a user account. Important attributes are account name, full name, contact email, password hash, and a list of recent login times. What kind of validation should I do in the constructor?

When creating a Java class for a user account with attributes like account name, full name, contact email, password hash, and a list of recent login times, it's important to include appropriate validations to ensure data integrity and security. Below is a suggested Java class with the required validations:

```
import ...;

public class UserAccount {
    private String accountName;
    private String fullName;
    private String contactEmail;
    private String passwordHash;
    private List<String> recentLoginTimes;
    // Constructor
    public UserAccount(String accountName, ...) {
        this.setAccountName(accountName);
        this.setFullName(fullName);
        ...
    }

    // Validation methods used by constructor
    private void setAccountName(String accountName) {
        if (accountName == null ||
            accountName.isEmpty() ||
            accountName.length() < 3 ||
```

```

        accountName.length() > 20)
    {
        throw new IllegalArgumentException(
            "Name must be between 3 and 20
characters.");
    }
    this.accountName = accountName;
}
private void setFullName(String fullName) { ... }
private void setContactEmail(String contactEmail)
{ ... }
private void setPasswordHash(String passwordHash)
{ ... }
private void setRecentLoginTimes(
    List<String> recentLoginTimes)
{ ... }

// Utility method to validate email format
private boolean isValidEmail(String email) { ... }

// Getters
public String getAccountName() {
    return accountName;
}
public String getFullName() { ... }
public String getContactEmail() { ... }
public String getPasswordHash() { ... }
public List<String> getRecentLoginTimes() { ... }
}

```

Validation Details

- **Account Name:** Must be between 3 and 20 characters, shouldn't be null or empty.
- **Full Name:** Shouldn't be empty and must be less than 50 characters.
- **Contact Email:** Must be a valid email format using a regular expression.
- **Password Hash:** Must be a 64-character string assuming a SHA-256 hash, shouldn't be null or empty.
- **Recent Login Times:** Initializes an empty list if the provided list is null to avoid `NullPointerException`.

[more details]

As is so often the case, the answer goes a little beyond the actual question. ChatGPT proposes the complete structure of the class, defines the `private` instance variables, and provides getter and setter methods for data access. The validation takes place in the setter methods. We're questioning this concept.

Prompt: Java: Using getters and setters only seems to complicate my code. What's the benefit?

In its response, ChatGPT explains that the chosen approach actually makes sense here. We therefore follow the suggestion and implement the class, including properties, getter and setter methods, and the constructor.

2.3.2 Output of the Object

We next want to equip the class with the `toString` method, which is particularly useful for tests because it allows us to easily output an object with all of its properties.

In the prompt, we ask ChatGPT not to output the code of the entire class each time, but only the code of the new `toString` method. We've greatly shortened the code outputs. If you want to view the entire code, just take a look at the sample files for this chapter.

Prompt: Please add a `toString` method to easily output the account data in a readable form. Don't answer with the entire code; only show the necessary changes.

Here are the necessary changes to add a `toString` method for the `UserAccount` class:

```
@Override
public String toString() {
    return "UserAccount{" +
        "accountName='" + accountName + '\'' +
        ...
        ", recentLoginTimes=" + recentLoginTimes +
```

```
        "},"";  
    }  
}
```

2.3.3 JSON Export and Constructor

Now we want to extend the class with a method for the JavaScript Object Notation (JSON) export. We also want to implement an additional JSON constructor. In the prompt, we ask for a solution that is as simple as possible without the use of external libraries.

Prompt: I want to export all attributes as a JSON string. Can you provide a simple method without using an external library?

To export the `UserAccount` object to a JSON string without using any external libraries, you can manually construct the JSON string. Here's how you can do it:

```
// Method to convert the object to JSON string  
public String toJson() {  
    StringBuilder json = new StringBuilder();  
    json.append("{");  
    json.append("\"accountName\":").  
        append(escapeJson(accountName)).  
        append("\",");  
    ...  
    return json.toString();  
}  
  
// Helper methods  
private String escapeJson(String value) { ... }  
private String listToJson(List<String> list) { ... }
```

Once we've tested the code, we want a JSON import function as a second constructor in the next prompt.

Prompt: What might a second constructor that expects a JSON string look like?

To create a second constructor that initializes a `UserAccount` object from a JSON string, you need to parse the JSON string and extract the necessary

fields. Because we're not using any external libraries, we'll manually parse the JSON string. Here is how you can implement the second constructor:

```
// Second constructor expecting a JSON string
public UserAccount(String jsonString) {
    this(parseJson(jsonString, "accountName"),
        ...
        parseJsonArray(jsonString,
            "recentLoginTimes"));
}
// Helper methods to parse JSON strings and arrays
private static String parseJson(
    String jsonString, String key) { ... }
private static List<String> parseJsonArray(
    String jsonString, String key) { ... }
```

2.3.4 Example: Bank Account

The next exercise is about mapping a bank account as a Java class. In the prompt, we first describe the task. But before we get started, we still have some basic questions. Which data types should we use? Should the properties be implemented as private or public variables?

Prompt: I have to write a Java class for a bank account. It stores the name of the account holder, the current balance, and a list of all previous transactions. Which data types should I use? Should I make my instance variables public or private?

To write a Java class for a bank account, you need to decide on appropriate data types for each attribute and determine the visibility of your instance variables. Here are the recommendations for both [more details].

Instance variables should generally be private to encapsulate the data and protect it from unauthorized access or modification. Public access should be provided through getter and setter methods.

```
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
```

```

public class BankAccount {
    private String accountHolderName;
    private BigDecimal currentBalance;
    private List<Transaction> transactionHistory;

    // constructor
    public BankAccount(String accountHolderName) { ...
}

    // methods
    public void deposit(BigDecimal amount) { ... }
    public void withdraw(BigDecimal amount) { ... }
    public String getAccountHolderName() { ... }
    public BigDecimal getCurrentBalance() { ... }
    public List<Transaction> getTransactionHistory() {
... }

    // nested class representing a transaction
    public static class Transaction {
        private BigDecimal amount;
        private String type;
        private String date;
        ...
    }
}

```

ChatGPT opts for private class variables, which is obvious in view of the question. During our tests, ChatGPT was also overzealous in providing methods for securely debiting and crediting money. In particular, the `withdraw` method refuses to debit more money than is currently available. The `deposit` method checks that the amount of the deposit is greater than 0.

We wanted to ask about these details in the next step, but unfortunately, this is bypassed. If you want to program efficiently and quickly, then ChatGPT's comprehensive answers are undoubtedly useful. If, on the other hand, you want to learn to program and contribute as much of your own code as possible, the overzealousness of AI tools is a hindrance.

2.3.5 Example: Sorting Photos Based on EXIF Data

The third example is formulated even more openly. The goal is a program to sort photos into directories according to the dates they were taken. The prompt describes in great detail what the program should be able to do and which library should be used to read the exchangeable image file format (EXIF) data. Once again, we try to make it explicitly clear to our AI tool that we don't want finished code, but a suggestion for organizing the program.

Prompt: I am trying to write a Java program that loops through all the JPEG files in a directory. The program uses `metadata-extractor` to read the EXIF data of each file. Images with valid EXIF data are then moved to a new directory `yyyy-mm`, where `yyyy` is the year of creation, `mm` is the month.

Please help me organize the code in a proper object-oriented way. Just provide a framework for the code, no methods yet.

Certainly! To organize your Java program in a proper object-oriented way, we can create a few classes to handle different responsibilities. Here's a framework with a brief explanation of each class's role:

- `JPEGProcessor`: Main class to initiate the process and handle the overall flow.
- `FileHandler`: Responsible for file operations like reading and moving files.
- `EXIFExtractor`: Uses `metadata-extractor` to read and parse EXIF data from JPEG files.
- `DirectoryOrganizer`: Manages the creation of new directories based on EXIF data.
- `Logger`: Handles logging of operations and errors.

[more details with code for all classes]

Of course, there are many ways to distribute the necessary code across several classes. The AI proposal summarized here is absolutely useful and a good starting point. What's annoying is that ChatGPT again can't resist the temptation to include a lot of sample code.

2.3.6 Example: Dealing with Singletons

When you learn a new programming language—regardless of whether it’s your first language or whether you can already program in other languages—you’ll always come across new features. You’ll see programming techniques in books and on the internet that you don’t understand straight away.

Pair programming with an LLM allows you to try out concepts in dialog mode and ask specific questions based on your task. The more information you provide about your level of knowledge and the initial situation, the better the AI can respond to your question.

In the following prompt, we assume that you’ve heard or read about the singleton pattern. Although the concept sounds useful, what does its implementation in Java look like?

Prompt: How can I realize the singleton pattern in Java?

ChatGPT interprets the question in the sense of determining how many possibilities there are to implement singletons in Java, and it presents six variants in its answer. We’ve only reproduced the first one.

In Java, the Singleton pattern ensures that a class has only one instance and provides a global point of access to it. There are several ways to implement a Singleton pattern in Java, each with its own advantages and disadvantages. Here’s a detailed look at the most common implementations.

Eager Initialization: This approach creates the instance when the class is loaded. It’s simple but may lead to resource wastage if the instance is never used.

```
public class Singleton {  
    // private static instance, so it can be accessed  
    // by only getInstance() method  
    private static final Singleton instance =  
        new Singleton();  
    // private constructor to prevent instantiation  
    from
```

```
// other classes
private Singleton() {}

// public method to provide access to the instance
public static Singleton getInstance() {
    return instance;
}
}
```

[more details: lazy initialization, thread-safe singleton, thread-safe singleton with double-checked locking, Bill Pugh singleton implementation, enum singleton]

ChatGPT overshoots the mark—as usual—and bombards you with details of six(!) implementation variants. That goes too far because you intend to use singletons to store a database connection centrally in the program. The concrete prompt leads to a much more concrete answer.

Prompt: Which variant is best if my program needs a single database connection?

If your program needs a single database connection, the Bill Pugh Singleton implementation is generally the best variant to use. [more details]

You trust the answer and refrain from comparing it with other implementation variants. Instead, in the next prompt, you ask whether it wouldn't be sufficient to equip your intended database class with a conventional constructor only.

Prompt: I intend to write a database class that contains all database functions I need for my program. What are the advantages of the Bill Pugh Singleton implementation over a traditional constructor?

Once you're finally convinced, you should further specify the objective in the next prompt. You specify which database system you want to use and ask the AI tool for a simple sample implementation.

Prompt: OK, let's use the Bill Pugh Singleton implementation. My database system is PostgreSQL. Show me a simple implementation. Be concise!